

基于统计检验与利润效益分析的生产优化问题研究

摘要

在当今快速发展的电子产品市场中,生产决策优化问题已成为制造业领域的关键研究议题。本研究聚焦于电子产品生产流程中的决策制定,采用统计学与运筹学的理论基础,综合考量了次品率、成本及售价等关键指标。以此为基础,本论文构建了一个以利润最大化为目标的数学模型。进一步地,灵活使用优化思想,对所构建的模型进行了系统的求解与分析。研究旨在为电子产品制造企业提供科学的决策支持,以提升生产效率并优化资源配置。

针对问题一,本文提出了一种基于正态分布近似法^[1]的抽样检测模型。该模型使用了统计学中的假设检验法,用二项分布描述抽样过程,综合考虑了显著性水平和抽样误差,同时分别考虑了在95%和90%的信度下,计算出最小样本量和相应的次品数量阈值,以决定是否接收供应商的零配件。本文同时利用python模拟在不同样本量下,次品率的变化情况,并与两个不同置信水平下的临界值进行比较。辅助判断样本量如何影响次品率的估计,以及如何根据置信水平做出质量控制决策。

针对问题二,研究了一个企业生产过程中涉及的质量控制问题。为了有效应对生产过程中的决策问题,本研究构建了一个以利润最大化为目标的多阶段决策模型。该模型综合考虑了检测成本、装配成本、调换损失以及拆解费用等多种成本因素,计算不同情况下的成本和收益,最终得出各阶段的最优决策方案,旨在为企业提供一个经济有效的生产策略。此外,为了进一步提升决策效率和精确度,本文还引入了状态压缩,并辅以剪枝,以优化整个决策过程,确保在复杂多变的生产环境中做出科学合理的管理决策。

针对问题三,扩展问题二的模型,针对企业在多零配件、多工序的复杂生产环境下的质量控制问题,建立了一个基于多阶段动态规划的精细化生产决策模型。该模型构建了一个多层次的生产网络,综合考虑多种成本因素,通过动态规划优化每一阶段的检测和拆解决策,最大化总利润。模型的求解结果为企业在生产各阶段提供了最优的质量控制策略和生产决策依据,有助于在复杂的生产环境中有效降低成本并提高产品质量。

针对问题四,本文采用置信区间,以95%信度为例,假设各次品率以正态分布方式分布在置信区间中。对于问题二的重新完成:采用遍历法,最终求出最优决策。对于问题四的重新完成:考虑到其次品率的数量较多,先求得当全部次品率为置信区间上限和下限时对应的最优决策,之后通过论证方式求出最优决策。

最后,本文对提出的模型进行全面的评价:本文的模型贴合实际,能合理解决提出的问题,具有实用性强,算法效率高等特点,该模型在其他类似生产也能使用。

关键词: 正态分布 生产决策 抽样检测 利润效益分析 01 规划^[2]

一、问题重述

1.1 问题背景

在现代电子产品制造业中，质量控制是确保消费者满意度和企业信誉的核心环节。随着科技的飞速发展，电子产品变得更加精密和复杂，对零配件的质量和生产过程的精确度提出了更高的要求。制造商在生产过程中必须做出一系列关键决策，这些决策不仅影响产品的最终质量，也直接关系到企业的经济效益和市场竞争力。

在生产线上，制造商需要决定是否对每个成品进行检测，或者直接将成品投放市场。此外，对于检测出的不合格成品，企业需要决定是进行拆解回收零配件，还是直接报废处理。这些决策需要在保证产品质量的同时，考虑成本效益和资源的可持续利用。

企业在追求高质量的同时，也必须考虑生产成本。这包括零配件的购买成本、检测成本、装配成本、市场售价以及不合格品的调换损失等。如何在控制成本的同时保证产品质量，是企业需要解决的问题，也是消费者对产品价格敏感性的直接体现。

1.2 问题提出

在电子产品的生产过程中，制造商面临着如何确保产品质量和控制成本的双重挑战。企业生产一种流行的电子产品，依赖于两种类型的零配件。这两种零配件的合格状态直接影响到最终成品的质量。若任何一个零配件不合格，则成品将不合格；即使两个零配件都合格，成品也可能因其他因素而不合格。对于检测出的不合格成品，企业面临报废或拆解的选择，后者虽可回收零配件，但会产生额外的拆解成本。

本问题涉及多方面的问题，包括零配件的检测、成品的装配和不合格品的处理。具体需要解决以下四个问题：

- 1) 设计一个经济有效的抽样检测方案，以在不同信度水平下判断零配件的次品率是否超出供应商声明的标称值。
- 2) 在已知零配件和成品次品率的情况下，制定生产过程中的检测、装配和不合格品处理策略，以最大化利润。
- 3) 针对包含多道工序和多个零配件的产品，建立模型以优化生产决策，考虑次品率、成本和售价等因素。
- 4) 考虑抽样误差对生产决策的影响，重新评估问题二和问题三的决策方案，并提出调整策略。

二、问题分析

2.1 对于问题一的分析：

首先，本文建立了一个基于二项分布的假设检验模型，用以描述抽样检测的过程。本文假设零配件的次品率为一个特定的标称值，并使用统计学中的假设检验来判断实际次品率是否显著偏离这一标称值。在这个过程中，针对两种不同的置信水平，本文分别计算出接收和拒收零配件的标准。

为了简化二项分布的计算，本文采用正态分布近似法，将实际次品率的分布用正态分布来近似。这种近似方法在样本量较大时非常有效。基于这个假设，本文推导出相应的公式，以确定在给定的置信水平下接受或拒收零配件的条件。进一步的计算过程中，本文考虑了样本大小的选择，使其满足在这两种情形下的指定拒收或接收概率要求。

通过这样的分析方法，本文为企业提供了在不同置信水平下的样本量和次品数量的参考值，从而有效指导零配件的质量控制决策。

2.2 对于问题二的分析：

在企业的生产过程中，为了提高产品质量并降低生产成本，需对采购的零配件（零配件 1 和零配件 2）及组装后的成品进行检测和处理。具体而言，企业需要在以下决策点上做出选择：

- 1) 零配件检测：选择是否检测零配件 1 和/或零配件 2，以筛除不合格品。
- 2) 成品质量控制：决定是否对成品进行检测，确保市场供应的产品质量。
- 3) 不合格成品处理：对不合格成品选择拆解回收或直接废弃，重新利用合格组件。
- 4) 市场反馈应对：提供不合格品调换，并重新进入处理流程。

本文将针对上述决策点，设计基于成本和收益的优化模型，并灵活使用剪枝策略做了一定的优化，分析不同决策下的最优方案，并给出具体的决策依据及相应的指标结果。

2.3 对于问题三的分析：

在扩展问题二的基础上，本文针对包含多个零配件和多道工序的复杂生产环境，构建了一个多阶段动态规划模型。该模型在每个生产决策点评估是否进行检测、选择检测方式及决定不合格成品的处理策略，如拆解回收。这些决策不仅即时影响成本和质量，也对后续生产环节产生连锁效应。

为精确描述这一决策流程，本文定义了一系列决策变量，关联不同生产阶段的检测与处理方法，并围绕利润最大化构建了优化模型。成本模型综合考量了采购、检测、装配、拆解及调换损失等成本因素。通过成本效益分析，本文确定了检测优先级，优化了决策过程，确保了在保证产品质量的同时，实现成本控制。此外，本文通过深入分析，将问题分解为零配件装配成半成品和半成品装配成成品两个主要部分，进一步细化为检测和拆解两个子问题。对于半成品 1 和 2，本文封装了检测与拆解过程为函数，减少了代码复杂性并增强了逻辑清晰度。特别地，引入了虚拟零配件 9 以简化半成品 3 的处理，其参数设置为零，以实现问题归一化。在成品装配的第二阶段，本文面临半成品数量和检测状态的不确定性，这些由第一阶段的结果决定且不可逆。

通过该模型的建立，本文的模型为复杂生产环境下的决策提供了科学依据，帮助企业在质量控制与成本优化间找到平衡点，为实际生产策略的制定提供了理论支持。

2.4 对于问题四的分析：

若次品率由抽样检测得到，则在不同信度下（本文以 95%信度为例）实际次品率所分布的置信区间会有所不同。可根据置信区间= $[\text{样本次品率}-Z \text{ 分数} \times \text{标准差}, \text{样本次品率}+Z \text{ 分数} \times \text{标准差}]$ 计算置信区间。若想求得最优决策，应考虑到所有次品率组合所对应的最优决策，并进行统计与分析。

对于问题二的重新完成：考虑到问题二中一共只有 3 个器件的次品率，所以可对次品率以一个合适的步长进行遍历，得到所有次品率组合下的最优决策，若某决策数量明显多于其他决策（由于“明显多于”，所以可忽略不同次品率为正态分布从而导致的次品率组合出现概率不同的情况），则其应为最终的最优决策。

对于问题三的重新完成：由于问题三中共有 12 个器件的次品率，采用遍历法将会导致计算量过大，所以先将置信区间的下限与上限代入至所有的次品率当中，之后发现二者对应的最优决策相同，再根据计算提出两条引理，最终论证得出最优决策。

三、模型假设与符号说明

3.1 模型基本假设

- 1) 零配件的质量检验是独立的，即抽样过程中检测一个零配件的结果不会影响另一个零配件的检测结果。
- 2) 检验方法遵循二项分布，因为每个零配件检测的结果只有合格或不合格两种可能。
- 3) 每种零配件的次品率是已知且固定的，不随时间或检测次数变化。
- 4) 半成品、成品的次品率是将正品零配件（或者半成品）装配后的产品次品率。
- 5) 问题三中成品被拆解只能被拆解成半成品，而不能拆解成零配件。

3.2 符号说明

表1 符号说明

符号	含义	单位
p_0	标称次品率	无
n	样本大小	件
Z	标准正态分布的临界值	无
p	实际次品率	无
C_{di}	零配件 <i>i</i> 的次品率	无
P_{pi}	零配件 <i>i</i> 的采购单价	元/件
C_{ti}	零配件 <i>i</i> 的检测成本	元/件
C_a	成品的组装成本	元/件
C_{tp}	成品的检测成本	元/件
C_d	成品的次品率	无
C_{dm}	不合格成品的拆解成本	元/件
C_{el}	不合格成品的调换损失	元/件
M	成品的市场价格	元/件
N_i	零配件 <i>i</i> 的采购数量	件
Q_p	组装后成品的数量	件
C_{profit}	净收入	元
C_{bi}	半成品 <i>i</i> 的次品率	无
$C_{purchase}$	零配件的采购成本	元
C_{text}	检测成本	元

这里只列出论文各部分通用符号，个别模型单独使用的符号在首次引用时会进行说明。

四、模型建立与求解

4.1 问题一模型建立与求解

4.1.1 问题一模型建立

在抽样检验中，正态分布近似法是一种常用的统计方法，适用于样本量较大时对二项分布进行近似计算。

1) 建立假设检验模型

首先，本文使用二项分布来描述抽样的过程：

零假设 H_0 ：次品率等于标称值，即 $p = 0.1$

备择假设 H_1 ：次品率不等于10%

在以下两种情形下，本文根据不同的置信水平来决定是否拒收或接收零配件：

情形 1:在 95%的置信水平下拒绝零配件，意味着在 95%的置信水平下拒绝 H_0 ，即认为次品率超过10%。

情形 2:在 90%的置信水平下接收零配件，意味着在 90%的置信水平下不拒绝 H_0 ，即认为次品率不超过10%。

2) 建立抽样检测模型

本文采用二项分布的正态近似来设计抽样方案。假设在样本大小为 n 的情况下，实际次品率 $p = \frac{X}{n}$ 服从二项分布：

$$X \sim \text{Binomial}(n, p) \quad (1)$$

其中， X 是样本中检测到的次品数量， p 是批次的实际次品率。由于当 n 较大时，二项分布可以用正态分布近似，因此本文采用正态近似的方法：

$$p \approx N\left(p, \frac{p(1-p)}{n}\right) \quad (2)$$

为了设计抽样方案，本文假设实际次品率 p 相对于标称次品率 p_0 存在偏离。根据这一假设，本文可以设置允许的次品数量范围，以满足在给定信度水平下接收或拒收批次零配件的要求。同时，为了满足问题中给定的信度要求，本文需要确定样本大小 n ，使得在两种情形下均能满足指定的拒收或接收概率。

针对情形 1，在 95% 信度下认定次品率超过标称值（拒收条件）。假设当 $p > p_0$ 时，拒收的概率为 95%。本文进一步假设 $p = p_0 + \delta$ ，其中 $\delta > 0$ 表示实际次品率超出标称值次品率的量。利用正态分布近似计算，本文得到以下公式：

$$Z_{0.95} \leq \frac{\delta}{\sqrt{\frac{p_0(1-p_0)}{n}}} \quad (3)$$

针对情形 2，在 90% 信度下认定次品率不超过标称值（接收条件）。假设当 $p \leq p_0$ 时，接收的概率为 90%。本文进一步假设 $p = p_0 - \delta$ ，其中 $\delta > 0$ 表示实际次品率低于标称次品率的量。利用正态分布近似计算，本文得到以下公式：

$$Z_{0.90} \leq \frac{\delta}{\sqrt{\frac{p_0 \cdot (1-p_0)}{n}}} \quad (4)$$

在上述两种情形下，根据相应的公式可以推导得出满足指定的信度要求的 n 值。进而可以求解出两种情况下的次品数量阈值 X_{th} 。

4.1.2 问题一模型求解与分析

1) 情形 1:

对于95%信度，查标准正态分布表^[2]可知， $Z_{0.95} = 1.645$ 。由题中条件 $p_0 = 0.1$ ，带入(3)式可得

$$\delta \geq 1.645 \times \sqrt{\frac{0.1 \times (1-0.1)}{n}} \quad (5)$$

将不等式两边平方并化简，整理可得：

$$n \geq \frac{1.645^2 \times 0.1 \times 0.9}{\delta^2} \quad (6)$$

为了保证检测的灵敏度，选择一个合理的 δ 值。综合考虑检测灵敏度、业务需求、行业标准 and 计算结果，本文假设 $\delta = 0.02$ （意味着认为实际次品率比标称值高出2%为显著偏离）。然后计算 n 值可得：

$$n \geq \frac{(2.706 \times 0.09)}{0.0004} = \frac{0.24354}{0.0004} \approx 609 \quad (7)$$

计算出最小样本量之后，根据

$$X_{th} \geq n \times p_0 + Z_{0.95} \times \sqrt{n \times p_0 \times (1-p_0)} \quad (8)$$

将 $\delta = 0.02$ ，代入得到：

$$X_{th} \geq 609 \times 0.1 + 1.645 \times \sqrt{609 \times 0.1 \times 0.9} \approx 73 \quad (9)$$

因此，情形1下最小样本量 n 大约为609，此时的次品数量阈值约为73个。当样本中检测到的次品数量超过73个时，可以认为实际次品率超过10%，从而拒收该批零配件。

2) 情形 2:

对于90%信度，查标准正态分布表可知， $Z_{0.90} = 1.28$ 。由题中条件 $p_0 = 0.1$ ，带入(4)式可得

$$\delta \geq 1.282 \times \sqrt{\frac{0.1 \times (1-0.1)}{n}} \quad (10)$$

同理计算 $n \geq 370$ 。根据

$$X_{th} \leq n \times p_0 - Z_{0.90} \times \sqrt{n \times p_0 \times (1-p_0)} \quad (11)$$

计算可得 $X_{th} \leq 30$

故情形 2 下最小样本量 n 大约为 370，此时的次品数量阈值约为 30 个。当样本中检测到的次品数量超过 30 个时，当样本中检测到的次品数量不超过 30 个时，可以认为实际次品率不超过10%，从而接收该批零配件。

4.1.3 问题一结果分析

本文同时利用 python 模拟和可视化在不同样本量下，次品率的变化情况，并与两个不同置信水平下的临界值进行比较。辅助判断样本量如何影响次品率的估计，以及如何根据置信水平做出质量控制决策。

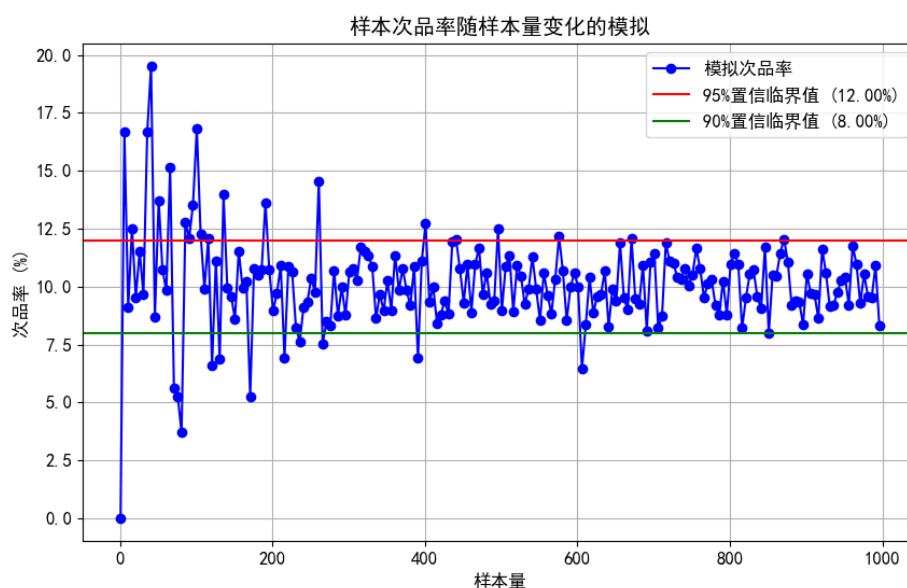


图1 模拟样本次品率随样本量的变化

在对模拟数据进行详细分析的基础上，进一步通过图一来直观展示样本量与次品率之间的关系。通过这一可视化手段观察到，当样本量增加至大约 300 个时，在90%的置信水平下，模拟得到的次品率接近于上文通过模型(4)计算得到的阈值。同样地，当样本量提升至大约 600 个时，模拟次品率在95%置信水平下的表现与模型(3)所预测的结果相吻合。这些观察结果不仅与通过数学推导得出的理论阈值一致，而且进一步验证了所采用的统计模型和计算方法在实际应用中的有效性和可靠性。至此，本文得出结论：

1) 情形 1:

在95%置信度下，假设实际次品率 p 高于标称次品率 p_0 2%的情况下计算得到的最小样本量为 **609**。次品数量阈值为 **73** 个。当样本中的次品数量超过 73 个时，拒收零配件。

2) 情形 2:

在90%置信度下，假设实际次品率 p 低于标称次品率 p_0 2%的情况下计算得到的最小样本量为 **370**。次品数量阈值为 **30** 个。当样本中的次品数量不超过 30 个时，接收零配件。

4.2 问题二模型建立与求解

4.2.1 问题二求解思路

模型的核心是对不同检测和处理策略进行评估，以找到最佳的决策组合。为此，将按照以下几个步骤求解：

1) 初始化参数：

零配件 1 和 2 的次品率、购买单价、检测成本，成品的次品率、装配成本、检测成本、市场售价以及不合格成品的调换成本、拆解费用的参数设定。

2) 状态定义：

使用一个 4 位二进制状态变量表示各阶段的决策，其中每一位对应一个决策点（是否对零配件和成品进行检测、是否对不合格成品进行拆解等）。巧妙利用剪枝的思想剔除不合理的状态优化。

3) 计算成本和收益：

对每种状态组合，计算其总成本，包括检测成本、拆解成本、换货损失等，同时计算相应的收益。图 2 简要介绍了针对问题二的决策框架。

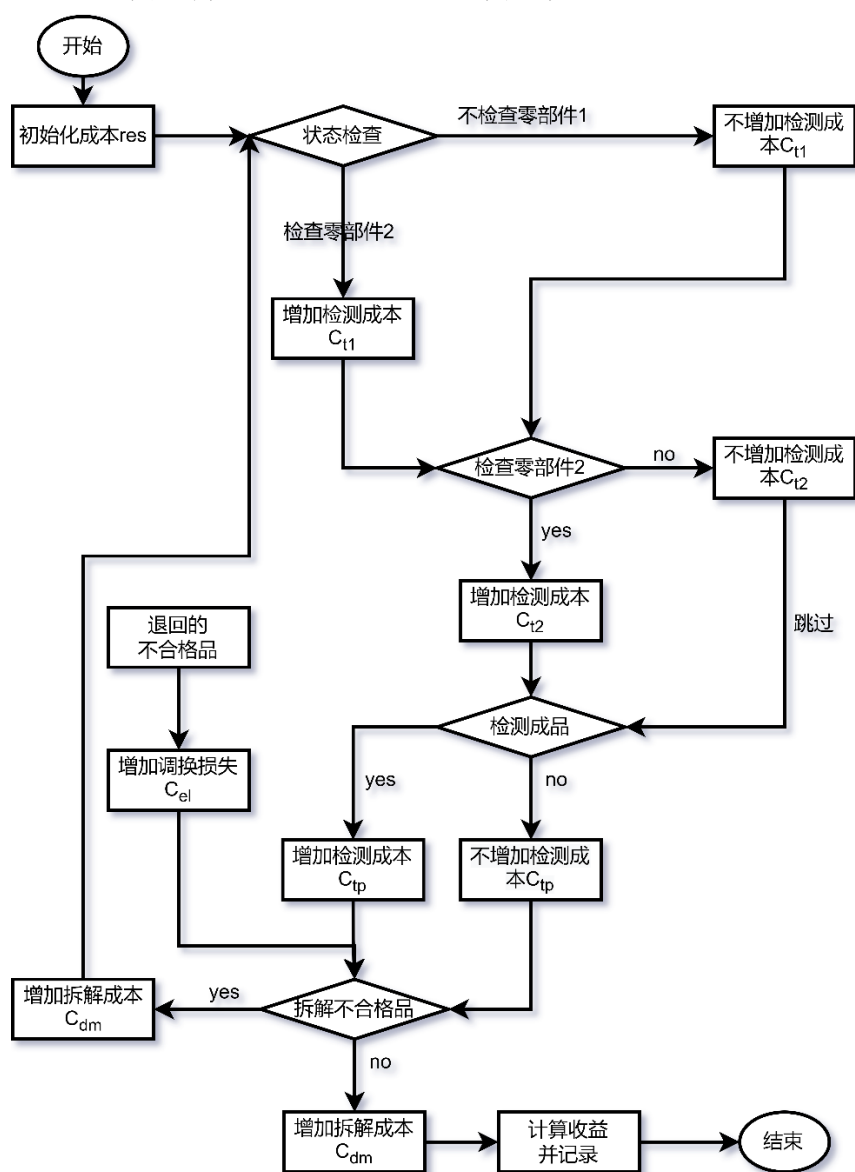


图2 零配件与成品质量控制决策流程图

4) 选择最优策略:

比较不同策略下的净收益，选取收益最大的策略作为最优方案。

4.2.2 问题二模型建立与求解

根据问题分析，模型通过定义四个决策阶段的状态来描述整个生产过程。利用 01 规划的思想，本文设置状态由一个 4 位二进制数字表示 $S \in \{0, 1\}^4$ ，总共有 16 种可能的状态组合。通过这种方式，每一种状态组合可以通过其对应的二进制数值所唯一确定，从而简化了决策过程的分析和记录。

1) 状态编码与决策映射

在本研究中，每个决策状态由一个四位二进制数表示，从而构成一个独特的状态标识。这种表示方法允许将每个状态映射到一个从 0 至 15 的数值，为每种可能的决策组合提供了一个明确的编号。

1、状态定义:

- a) 位 1 (最低位): 是否对零配件 1 进行检测 (1 表示检测, 0 表示不检测)
- b) 位 2: 是否对零配件 2 进行检测
- c) 位 3: 是否对装配好的成品进行检测
- d) 位 4 (最高位): 是否对不合格的成品进行拆解处理

但当零件之一或都未检测，同时进行拆解，由于装配不合格的零件成品一定不合格且无法剔除不合格零件，将会导致生产流程陷入无效的“装配-检测/调换-拆解”循环。

表 2 展示了全部的 16 种状态编码，并对无效状态进行了备注:

表2 问题二的决策状态编码表

state	零件 1 检测	零件 2 检测	成品检测	成品拆解	备注
0	0	0	0	0	
1	1	0	0	0	
2	0	1	0	0	
3	1	1	0	0	
4	0	0	1	0	
5	1	0	1	0	
6	0	1	1	0	
7	1	1	1	0	
8	0	0	0	1	与实际不符
9	1	0	0	1	与实际不符
10	0	1	0	1	与实际不符
11	1	1	0	1	
12	0	0	1	1	与实际不符
13	1	0	1	1	与实际不符
14	0	1	1	1	与实际不符
15	1	1	1	1	

后续模型求解过程简化为关注四位二进制数字。通过查阅表 2 的决策状态编码表对编码进行解码，从而直接推导出相应的决策方案。

2、四个决策点如下：

a) 零配件检测

状态的第一位表示是否对零配件 1 进行检测，第二位表示是否对零配件 2 进行检测。若对某种零配件检测，检测出的不合格品将被丢弃，否则直接进入装配环节。

b) 成品检测

状态的第三位表示是否对装配后的成品进行检测。若检测，则检测不合格的成品进入下一阶段处理，否则直接进入市场销售环节。

c) 成品拆解处理

状态的第四位表示是否对检测不合格的成品进行拆解处理。若拆解，则拆解后的零配件将重新进入零配件检测阶段，否则直接丢弃。

d) 退货处理

若用户购买的成品为不合格品，将产生调换损失，并对退回的不合格品执行阶段 3。

2) 决策模型描述

给定状态 $S = (s_4, s_3, s_2, s_1)$ ，考虑在这一状态下可执行的各种决策和它们的影响，从而定义以下决策和相应的数学表达式：

两种零配件的采购成本：

$$C_{\text{purchase}} = P_{p1} \times N_1 + P_{p2} \times N_2 \quad (12)$$

检测成本：

若 $s_1 = 1$, 零配件 1 的检测成本为 $C_{t1} \times N_1$

若 $s_2 = 1$, 零配件 2 的检测成本为 $C_{t2} \times N_2$

若 $s_3 = 1$, 成品的检测成本为 $C_{tp} \times Q_p$

总检测成本 C_{test} 为：

$$C_{\text{test}} = s_1 \cdot C_{t1} \times N_1 + s_2 \cdot C_{t2} \times N_2 + s_3 \cdot C_{tp} \times Q_p \quad (13)$$

零配件 1 有效数量为：

$$N_1' = N_1 \times (1 - s_1 \cdot C_{d1}) \quad (14)$$

零配件 2 有效数量为：

$$N_2' = N_2 \times (1 - s_2 \cdot C_{d2}) \quad (15)$$

组装后成品的数量为：

$$Q_p = \min(N_1', N_2') \quad (16)$$

特别的，组装后成品的次品率会根据零件的检测情况发生变化，记为 C'_d ，其不同情况下的表达式如式（17）所示：

$$C'_d = \begin{cases} C_{di} + C_d - C_{di}C_d & (i \text{ 为 } S_1S_2 \text{ 中为 } 0 \text{ 的编号}) S_1 = 1 \text{ 或 } S_2 = 1 \\ C_d & S_1 = 1, S_2 = 1 \\ 1 - (1 - C_{d1})(1 - C_{d2})(1 - C_{di}) & S_1 \neq 1, S_2 \neq 1 \end{cases} \quad (17)$$

若 $s_3 = 1$ ，成品的有效数量为：

$$Q_{\text{valid}} = Q_p \times (1 - C_d) \quad (18)$$

若 $s_4 = 1$ ，则对不合格成品进行拆解处理，成本为：

$$C_{\text{rework}} = Q_p \times C_{dm} \times C'_d. \quad (19)$$

否则

$$C_{\text{rework}} = Q_p \times C_{el} \times C'_d \quad (20)$$

总成本 C_{total} 为：

$$C_{\text{total}} = C_{\text{purchase}} + C_{\text{test}} + Q_p \times C_a + C_{\text{rework}} \quad (21)$$

总收益 R 为：

$$R = Q_{\text{valid}} \times M \quad (22)$$

净收益(目标函数)：

$$C_{\text{profit}}(S) = R - C_{\text{total}} \quad (23)$$

3) 模型求解

目标是找到一个状态 $S = (s_4, s_3, s_2, s_1)$ 使得净收益 $C_{\text{profit}}(S)$ 最大化。通过遍历所有可能的状态组合 S （从 0000 到 1111，共 16 种），计算每种状态下的总成本和净收益，并选择净收益最大的状态作为最优策略。

4.2.3 问题二结果分析

1) 结果展示

本节深入探讨问题二的求解成果。通过应用前述数学模型，成功计算了不同决策情境下的净收益。为了使这些复杂的数据更易于理解和分析，本文同时利用 Mathematica 软件绘制了一系列散点图，以辅助在众多方案中识别最优选择。

图 4 至图 9 分别展示了六种不同情形下各决策方案的净收益情况图 4 至图 9 分别展示了六种不同生产情况下的决策方案和相应的净收益。这些图表中，每个点代表一个特定的决策方案，其在垂直轴上的位置表示该方案的净收益值。通过比较这些点的位置，可以直观地看出哪些决策方案在特定的生产情况下能够带来更高的经济效益。

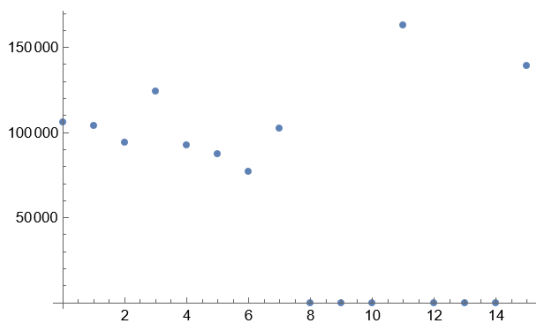


图3 情形1 各决策方案的净收益

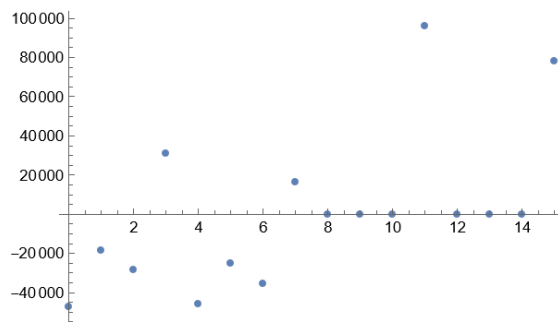


图4 情形二各决策方案的净收益

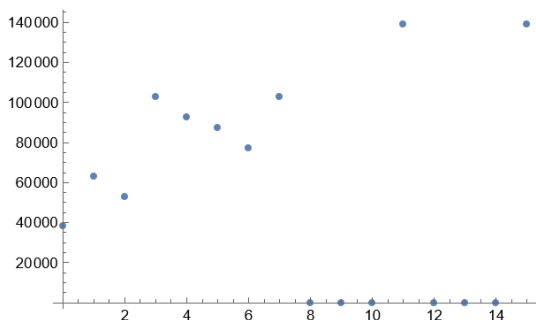


图5 情形三各决策方案的净收益

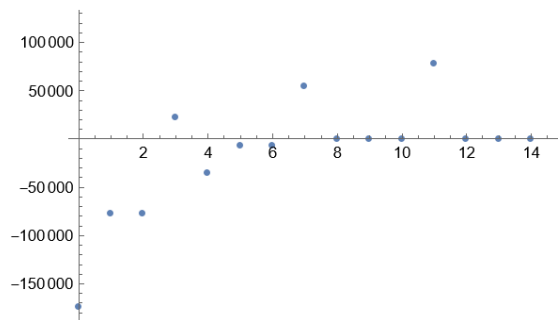


图6 情形四各决策方案的净收益

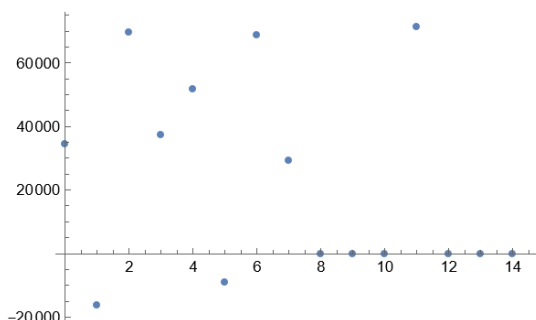


图7 情形五各决策方案的净收益

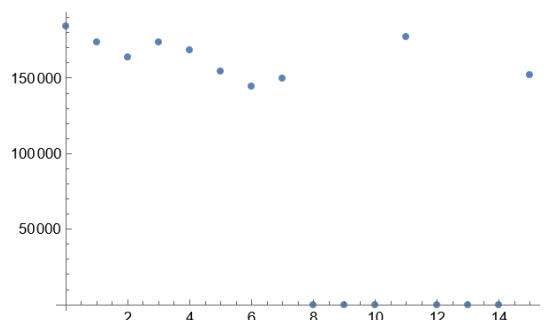


图8 情形六各决策方案的净收益

此外，通过模型分析求解，确定了每种情形下的最优决策方案及其相应的决策状态编码：

表3 问题二决策状态编码展示

情形	净收益/元	决策状态编码
1	163000	1011
2	96000	1011
3	139000	1011
4	118000	1111
5	71333.3	1011
6	184218	0000

通过对决策状态编码的解码，明确了每个情形下的最优决策方案。这些方案详细描述了在不同情形下应采取的具体措施，包括零配件的检测、成品的检测与否以及不合格成品的处理策略。以下是具体的决策方案：

表4 问题二决策方案展示

情形	净收益/元	决策方案
1	163000	检测零件 1、2；不检测成品；拆解不合格成品
2	96000	检测零件 1、2；不检测成品；拆解不合格成品
3	139000	检测零件 1、2；不检测成品；拆解不合格成品
4	118000	检测零件 1、2；检测成品；拆解不合格成品
5	71333.3	检测零件 1、2；不检测成品；拆解不合格成品
6	184218	不检测零件 1、2；不检测成品；不拆解不合格成品

2) 结果分析

从结果中可以看出，不同的生产情形下，最优决策方案有所不同。

在**情形 1**中，由于零配件的次品率相对较低，且检测成本不高，因此选择检测以降低不合格成品的风险。成品不进行检测，可能是因为装配后次品率仍可接受，且市场售价较高，使得不合格品的调换损失和拆解费用相对于市场售价而言较小。

在**情形 2**中，尽管零配件的次品率增加到 20%，但检测成本相对较低，因此检测零配件以减少不合格成品的产生是合理的。成品不检测的决策可能基于成本效益分析，考虑到次品率和市场售价，不合格品的直接成本可能低于检测成本。

在**情形 3**中，调换损失的增加可能意味着市场对不合格品的容忍度降低，或者不合格品对企业声誉的影响更大。尽管如此，检测零配件的策略仍然有效，因为通过减少不合格成品的数量，可以避免更高的调换损失。

在**情形 5**中，尽管零配件 1 的检测成本增加，但由于零配件 2 的检测成本较低，整体检测成本仍然可控。成品不检测的决策可能是基于次品率和市场售价的权衡，考虑到较低的调换损失，不合格品的市场影响较小。

可见，通过检测零配件来确保质量，并且避免成品检测的成本，是一个有效的策略。同时，拆解不合格成品回收零配件，这是提高净收益的一个关键因素。尽管零配件 1 和零配件 2 的次品率不同，但最优策略都是检测这两个零配件。这表明，当检测成本相对较低时，通过检测来降低不合格成品的风险是划算的。

在**情形 4**中，最优策略变为检测零配件和成品，检测成本显著降低，使得对所有零配件和成品进行检测成为更经济的选择。这可能是由于检测技术的进步或成本的降低，使得全面检测策略在成本上更具吸引力。

而在**情形 6**中，最优策略是不检测任何零配件和成品且不拆解不合格成品。零配件的次品率很低，使得检测的必要性降低。同时，成品的次品率也低，市场售价较高，调换损失和拆解费用相对较高，这可能导致企业选择不检测成品，以避免额外的检测和处理成本。

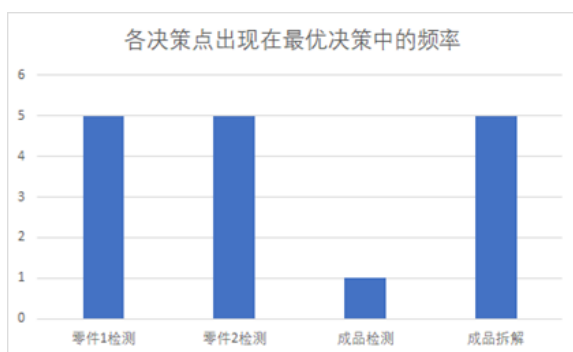


图9 各决策点出现在最优决策中的出现频率

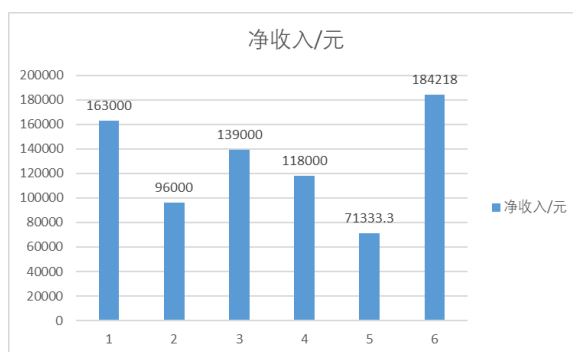


图10 各情形下的净收入

在对不同生产情形下的最优决策方案进行深入分析，并结合图 10 和图 11 所示的数据，本文针对电子产品生产策略提出以下建议：

a) **低次品率与低检测成本**（情形 1）：在这种情况下，由于次品率较低，检测成本也相对较低，因此选择对零配件进行检测是合理的。这种策略可以有效降低不合格成品的风险，同时由于次品率低，成品的总体质量较高，因此不检测成品是可行的。

b) **高次品率与低检测成本**（情形 2 与情形 5）：即使零配件的次品率较高，但由于检测成本仍然可控，检测零配件以减少不合格成品的产生仍然是一个有效的策略。这种策略有助于通过早期识别和排除次品来优化整体生产质量。

c) **高市场售价与低调换损失**（情形 1 与情形 2）：在市场售价较高的情况下，即使成品存在次品，其市场售价也足以覆盖调换损失和拆解费用。因此，企业应倾向于不进行成品检测，以节省检测成本并快速进入市场。

d) **高调换损失**（情形 3）：当调换损失显著增加时，这可能反映了市场对不合格品的低容忍度或对企业声誉的潜在负面影响。在这种情况下，即使检测成本较高，全面检测策略也变得经济上可行，以避免高昂的调换损失。

e) **成本效益权衡**（情形 5 与情形 6）：在某些情况下，尽管零配件的检测成本增加，但整体检测成本仍然可控。企业需要进行成本效益分析，权衡检测成本与潜在的市场损失。在情形 6 中，由于零配件的次品率很低，市场售价较高，调换损失和拆解费用相对较高，企业可能选择不检测成品，以避免额外的检测和处理成本。

f) **全面检测**（情形 4）：随着检测费用和成本的降低，对所有零配件和成品进行检测成为可能。这种策略有助于提高产品质量，减少市场风险，并可能提高企业的市场竞争力。

4.3 问题三模型建立与求解

4.3.1 问题三求解思路

在问题二的求解过程中，本文采用了**枚举**和**剪枝**的方法。然而，在审视问题三时，发现直接枚举的方法并不适用。因此，本文对决策过程进行了深入分析，发现可以将问题分解为两个相互关联但有所区别的部分：零配件装配成半成品和半成品装配成成品。进一步细分子问题可能产生多达 32 或 16 种情况，这并非本文所期望的简化方式。通过对子问题的决策过程进行持续剖析，本文最终将每个子问题划分为两个有联系的部分：零配件的检测、半成品的检测与拆解。

1) 第一个子问题

以半成品 1 的装配为例，将其分为零配件的检测、半成品的检测与拆解两部分，将半成品的检测与拆解封装成函数以减少代码量并增强可读性、逻辑性。对于半成品的检测与拆解函数的参数，除了基础参数外，本文发现零配件的检测只会影响到半成品的数

量与次品率，而这两个参数足以支撑求解半成品的检测与拆解。本文将问题划分为以下步骤：

- a) 零配件的检测，涉及三个零配件的八种检测情况。
- b) 根据检测结果，确定半成品的数量和总次品率。
- c) 对半成品进行检测与拆解，封装为函数以简化代码并提高可读性和逻辑性。

特别地，半成品 2 与半成品 1 的处理过程完全相同。为了简化问题求解，本文引入了一个虚拟零配件 9 作为半成品 3 的第三个组装零配件，其参数（次品率、购买单价、检测成本）均设为 0，以实现问题归一化而不影响最终结果。

2) 第二阶段

即成品的装配过程中，半成品与成品的关系类似于零配件与半成品的关系。本文同样将问题划分为半成品的检测、成品的检测与拆解两部分。问题的复杂之处在于两点，半成品的初始数量不再是已知的，而是由第一阶段的结果确定，且半成品的检测情况是不可逆的（指不能改已检测为未检测）。

4.3.2 问题三模型建立

1) 决策状态的设置

针对问题三扩展问题二的决策变量编码方法，将多零配件、多工序的每一个决策点映射为一个独特的决策编码。这一过程涉及将整个生产流程划分为一系列决策点，并对每个决策点定义相应的状态编码。

首先，将整个生产流程中的多个工序和零配件划分为以下几类决策点：

- a) 第一类：
零配件 1 至零配件 8 的检测决策
- b) 第二类：
半成品 1 至半成品 3 的检测和拆解决策
- c) 第三类：
成品的检测和拆解决策

然后再根据产品组装流程以及设置的决策状态的，再将问题分为下列四个决策阶段如图 11 所示：

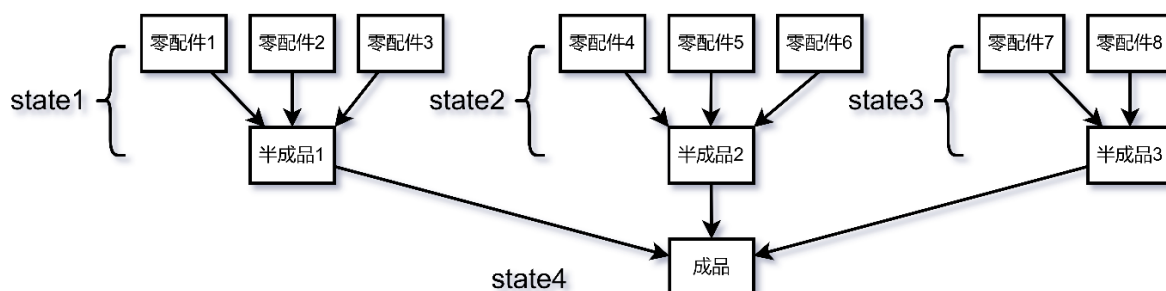


图11 产品组装流程及状态映射

定义状态编码 state1、state2、state3、state4 分别代表半成品 1、2、3 和成品的状态编码，而 s_1, s_2, s_3, s_4, s_5 表示其二进制表示的最低 1 位到 5 位（具体含义参见前文状态编码表）。

对于不同的决策点，有不同的决策状态编码

表5 state1 决策状态编码表

st at el	零件 3 检 测	零件 2 检 测	零件 1 检 测	半成 品 1 拆解	半成 品 1 检测	备注	sta te1 2	零件 3 检 测 3	零件 2 检 测 4	零件 1 检 测 5	半成品 1 拆解 6	半成品 1 检测 7	备注 8
0	0	0	0	0	0		16	1	0	0	0	0	
1	0	0	0	0	1		17	1	0	0	0	1	
2	0	0	0	1	0	与实际不符	18	1	0	0	1	0	与实际不符
3	0	0	0	1	1	与实际不符	19	1	0	0	1	1	与实际不符
4	0	0	1	0	0		20	1	0	1	0	0	
5	0	0	1	0	1		21	1	0	1	0	1	
6	0	0	1	1	0	与实际不符	22	1	0	1	1	0	与实际不符
7	0	0	1	1	1	与实际不符	23	1	0	1	1	1	与实际不符
8	0	1	0	0	0		24	1	1	0	0	0	
9	0	1	0	0	1		25	1	1	0	0	1	
10	0	1	0	1	0	与实际不符	26	1	1	0	1	0	与实际不符
11	0	1	0	1	1	与实际不符	27	1	1	0	1	1	与实际不符
12	0	1	1	0	0		28	1	1	1	0	0	
13	0	1	1	0	1		29	1	1	1	0	1	
14	0	1	1	1	0	与实际不符	30	1	1	1	1	0	
15	0	1	1	1	1	与实际不符	31	1	1	1	1	1	

(state2 的决策状态编码与表 3 类似，不再赘述)

表6 state3 决策状态编码表

state3	零件 8 检测	零件 7 检测	半成品 3 拆解	半成品 3 检测	备注
0	0	0	0	0	
1	0	0	0	1	
2	0	0	1	0	与实际不符
3	0	0	1	1	与实际不符
4	0	1	0	0	
5	0	1	0	1	
6	0	1	1	0	与实际不符
7	0	1	1	1	与实际不符
8	1	0	0	0	
9	1	0	0	1	
10	1	0	1	0	与实际不符
11	1	0	1	1	与实际不符
12	1	1	0	0	
13	1	1	0	1	
14	1	1	1	0	
15	1	1	1	1	

表7 state4 决策状态编码表

state4	成品拆解	成品检测
0	0	0
1	0	1
2	1	0
3	1	1

为了具体分析，本文假设所有零配件的初始数量为 1，以简化模型构建。

由于零配件 1,2,3 与零配件 4,5,6 和半成品 1 与半成品 2 参数完全相同，故根据对称性可以直接得出最优决策中 state1=state2，简化计算。

2) 第一阶段（零配件装配成半成品）

所求目标量如下：

a) 半成品的数量 Q_b

$$Q_b = \begin{cases} 1 - \max(C_{di}, C_{dj}, C_{dk}) & (\text{零配件 } i, j, k \text{ 已检测}) \\ 1 - \max(C_{di}, C_{dj}) & (\text{零配件 } i, j \text{ 已检测}) \\ 1 - C_{di} & (\text{零配件 } i \text{ 已检测}) \\ 1 & (\text{无零配件检测}) \end{cases} \quad (24)$$

b) 当前数量半成品的总次品率 C'_b （其中 C_b 为半成品的基础次品率）

$$C'_b = \begin{cases} C_b & (\text{零配件 } i, j, k \text{ 已检测}) \\ C_{di} + C_b - C_{di} \cdot C_b & (j, k \text{ 检测}) \\ 1 - (1 - C_{di}) \cdot (1 - C_{dj}) \cdot (1 - C_b) & (k \text{ 检测}) \\ 1 - (1 - C_{di}) \cdot (1 - C_{dj}) \cdot (1 - C_{dk}) \cdot (1 - C_b) & (\text{无检测}) \end{cases} \quad (25)$$

c) 装配出当前数量半成品的成本 C_{bcost}

其他数学量的定义：

$$C_{purchase} = \sum P_{pi} \quad (26)$$

$$C_{test} = \sum C_{ti} \quad (27)$$

由此完成零配件的检测

进入半成品装配、检测与拆解：

若 $s1_{state1} = 1$ ：

$$\begin{aligned} C_{test} + &= C_{tb} * Q_b \\ Q_b &= \begin{cases} Q_b & s2_{state1} = 1 \\ Q_b * (1 - C'_b) & s2_{state2} = 0 \end{cases} \\ C'_b &= 0 \end{aligned} \quad (28)$$

在本研究中，特别关注了当状态编码为 $state1=11111$ （对于半成品 1）和 $state1=01111$ （对于半成品 3）时的特殊情况。在这些特定情形下，即使所有组装的零配件均为合格品，由于半成品的基础次品率，依然不可避免地会产生不合格的半成品。这一现象引出了一个关键问题：在实际生产过程中，不合格品的检测、拆解和重新装配将形成一个理论上无限循环的过程。故在基础数量足够庞大的程度下本文近似可以用等比数列计算成本：

$$\begin{aligned} C_u &= Q_b * C'_b * (C_{ba} + C_{tb} + C_{dm}) * (1 / (1 - C_b)) \\ Q_b &= Q_b \\ C'_b &= 0 \end{aligned} \quad (29)$$

综上，半成品成本函数

$$C_{bcost} = C_{purchase} + C_{test} + Q_b * C_{ba} + C_u \quad (C_{ba} \text{ 为半成品装配费用}) \quad (30)$$

3) 第二阶段（即子问题半成品装配成品）

本文的目标函数为盈利 C_{profit}

成品数量：

$$Q_p = \begin{cases} \min(Q_{bi} * (1 - C'_{bi}), Q_{bj} * (1 - C'_{bj}), Q_{bk} * (1 - C'_{bk})) & (\text{半成品 } i, j, k \text{ 已检测}) \\ \min(Q_{bi} * (1 - C'_{bi}), Q_{bj} * (1 - C'_{bj}), Q_{bk}) & (\text{半成品 } i, j \text{ 已检测}) \\ \min(Q_{bi} * (1 - C'_{bi}), Q_{bj}, Q_{bk}) & (\text{半成品 } i \text{ 已检测}) \\ \min(Q_{bi}, Q_{bj}, Q_{bk}) & (\text{无半成品检测}) \end{cases} \quad (31)$$

当前数量成品（即第一批）的总次品率：

$$C'_d = \begin{cases} C_d & (\text{半成品 } i, j, k \text{ 已检测}) \\ C_{bi} + C_d - C_{bi} \cdot C_d & (j, k \text{ 检测}) \\ 1 - (1 - C_{bi}) \cdot (1 - C_{bj}) \cdot (1 - C_d) & (k \text{ 检测}) \\ 1 - (1 - C_{bi}) \cdot (1 - C_{bj}) \cdot (1 - C_{bk}) \cdot (1 - C_d) & (\text{无检测}) \end{cases} \quad (32)$$

特别的，由于半成品可能已被检测过，所以 $C_{test} = \sum C_{ti}$ 要避免重复计算半成品检测成本，论文不过多赘述。

由此完成半成品的检测，进入成品装配、检测与拆解：

若 $s1_{state4} = 1$ ：

$$\begin{aligned} C_{test} &+ = C_{tp} * Q_p \\ Q_p &= \begin{cases} Q_p & s2_{state4} = 1 \\ Q_p * (1 - C'_d) & s2_{state4} = 0 \end{cases} \\ C'_d &= 0 \end{aligned} \quad (33)$$

特别地，对于 $state4=11111$ 的情况，由于成品的基础次品率，即使半成品都是合格的，但依旧会装配出不合格成品，故成品检测，拆解，装配这一过程理论上会无限进行下去，在基础数量足够庞大的程度下本文近似可以用等比数列计算成本：

$$C_u = Q_p * C'_d * (C_a + C_{tp} + C_{dm}) * (1/(1 - C_d)) \quad (34)$$

对于 $state4=11110$ 的情况，与 $state4=11111$ 同理，成品调换，拆解，装配这一过程理论上也会无限进行下去：

$$C_u = Q_p * C'_d * (C_a + C_{el} + C_{dm}) * (1/(1 - C_d)) \quad (35)$$

综上，成品成本函数：

$$C_{cost} = \sum C_{bcost} + C_{test} + Q_p * C_a + C_u \quad (36)$$

成品盈利函数：

$$C_{profit}(state1, state2, state3, state4) = Q_p * M - C_{cost} \quad (37)$$

4.3.3 问题三结果分析

1) 结果展示

本文模型得出的最优策略是：

零配件、半成品全面检测，半成品都拆解，成品拆解但不检测

对于 1 套零配件 1-8，

盈利为 54.2 元。

2) 结果分析

从经济性角度进行分析：

a) 零配件：

考虑到零配件的次品率较高，而检测费用相对较低，因此从成本效益的角度来看，对所有零配件进行检测是经济合理的。这样可以在采购阶段就剔除次品，避免后续的维修或更换成本。

b) 半成品：

对于半成品，其次品率同样较高，但检测费用较低，并且与零配件相比，拆解半成

品的费用相对较低，接近于零配件的购价成本。因此，从经济性出发，对所有半成品进行全面检测，并在检测后进行拆解，以确保最终产品的质量。

c) 成品：

在成品阶段，次品率虽然存在，但调换费用相比于检测费用通常较低。因此，从经济性角度考虑，成品不进行检测，但需要在调换后进行拆解，以确保成品的质量符合标准。这样可以避免因检测成本过高而增加的额外费用，同时通过拆解检查能有效控制成品的次品率。

综上所述，通过这种分阶段的检测和拆解策略，可以在保证产品质量的同时，最大化成本效益，优化整体生产流程。

4.4 问题四的模型建立与分析

4.4.1 问题四的模型建立

对于问题四，若次品率由抽样检测的结果得到，则实际次品率不能直接准确得出。在信度不同时，实际次品率应位于不同的置信区间中（假设实际次品率分布为正态分布），本文将以 95%信度为例，对问题四做出解答：

首先确定置信区间：先求出 95%信度下对应的 Z 分数，再根据问题一中提及的 n（样本大小）计算标准差 $\sigma = \sqrt{\frac{p(1-p)}{n}}$ （以抽样检测所得次品率代替实际次品率）。

最后利用公式：置信区间 $U = [p - Z * \sigma, p + Z * \sigma]$ 求出置信区间（若某次品率位于置信区间之外，则其出现概率过小，可忽略其影响），不妨设置信区间下限为 a，上限为 b，则 $U = [a, b]$

下表为次品率不同时对应的置信区间

表8 次品率对应的置信区间

次品率	0.05	0.1	0.2
置信区间	[0.032,0.068]	[0.076,0.124]	[0.168,0.232]

4.4.2 对于问题二的重新求解

将零配件，成品的次品率在 U 中采用一个合适的步长进行遍历（以 $\frac{1}{8}(b-a)$ 为例），得到所有次品率组合下的最优决策，若某决策数量明显多于其他决策（由于“明显多于”，所以可忽略不同次品率为正态分布从而导致的次品率组合出现概率不同的情况），则其应为最终的最优决策

不同情况的不同次品率组合对应的所有决策见支撑材料“问题四 结果（文件夹）”。

表 9 为各情况的最优决策：

表9 重解问题二的决策状态编码展示

情况	1	2	3	4	5	6
最优决策	11	11	11 或 15	15	11	0

表10 重解问题二的决策方案展示

情形	决策方案
1	检测零件 1、2；不检测成品；拆解不合格成品
2	检测零件 1、2；不检测成品；拆解不合格成品
3	检测零件 1、2；不检测成品；拆解不合格成品或全部操作
4	检测零件 1、2、成品；拆解不合格成品
5	检测零件 1、2；不检测成品；拆解不合格成品
6	不检测零件 1、2、成品；拆解不合格成品

注：情况 3 的最优决策是由于在所有次品率组合对应的最优决策中，决策 11 和决策 15 的数量相同，并且之后通过编程验证知：当成品次品率为 0.1 时，决策 11 和决策 15 均为最优策略；当成品次品率大于 0.1 时，决策 15 为最优策略；当成品次品率小于 0.1 时，决策 11 为最优策略。

4.4.3 对于问题三的重新求解

令 $C_{di} = a (i = 1 \cdots 8)$ 、 $C_{bi} = a$ 、 $C_{bj} = a$ 、 $C_{bk} = a$ 、 $C_d = a$ ，可求得此时的最优决策为“31 31 15 2”（即为所有零配件均检测、所有半成品均检测并拆解、成品拆解但不检测），再令 $C_{di} = b (i = 1 \cdots 8)$ 、 $C_{bi} = b$ 、 $C_{bj} = b$ 、 $C_{bk} = b$ 、 $C_d = b$ ，得到的最优决策也为“31 31 15 2”。

下面将论证对于所有次品率 U 中的次品率组合，决策“31 31 15 2”均为最优决策：

两个基本引理：

引理 1：若某器件（包括零配件、半成品、成品）的次品率为 x ，此时的最优决策包含该器件检测（或拆解），则当该器件的次品率大于 x 时，最优决策应继续包含该器件检测（或拆解）

引理 2：若某器件（包括零配件、半成品、成品）的次品率为 x ，此时的最优决策包含该器件不检测（或不拆解），则当该器件的次品率小于 x 时，最优决策应继续包含该器件不检测（或不拆解）

具体论证：

1) 若 $C_{di} \in U$ ，则其 $C_{di} > a$ ，根据引理 1，此时的最优决策应包含零配件 i 检测。
($i = 1 \cdots 8$)

2) 若 C_{bi} 、 C_{bj} 、 $C_{bk} \in U$ ，则其次品率大于 a ，根据引理 1，此时的最优决策应包含半成品 i 、 j 、 k 检测并拆解。

3) 若 $C_{di} \in U$ ，则 $a < C_d < b$ ，根据引理 1，此时的最优决策应包含成品拆解；根据引理 2，此时的最优决策应包含成品不检测。

综上所述：当 $C_{di} = a (i = 1 \cdots 8)$ 、 C_{bi} 、 C_{bj} 、 C_{bk} 、 $C_d \in U$ 时，对于所有的次品率组合，决策“31 31 15 2”均为最优决策。

补充与拓展：可知当 C_{di} ($i=1\cdots 8$)、 C_{bi} 、 C_{bj} 、 $C_{bk}>a$ ， C_d 持续增大直到大于某一数字时，最优决策一定会由决策“31 31 15 2”变为决策“31 31 15 3”，对比以上两种决策，发现前者比后者多出调换费用，后者比前者多出检测费用，二者其余数据均一样。于是假设 C_d 的临界点为 x ，令 $x*C_{el}=C_{tp}$ 可解出 $x=0.15$ 。

所以，当 C_{di} ($i=1\cdots 8$)、 C_{bi} 、 C_{bj} 、 $C_{bk}>a$ ， $a<C_d<x$ 时，最优决策为决策“31 31 15 2”；当 C_{di} ($i=1\cdots 8$)、 C_{bi} 、 C_{bj} 、 $C_{bk}>a$ ， $x<C_d$ 时，最优决策为决策“31 31 15 3”。

五、模型评价与推广

5.1 模型的优点

1) 模型性能卓越：

本文成功构建了一个高精度决策模型，并通过精确的剪枝操作消除了不合理的分支，确保了模型的效率与准确性。

2) 全面考虑与精简决策：

在模型设计过程中，本文充分考虑了所有可能的情况，并采纳了归一化策略，如将问题三中半成品 1 和 2 的决策进行统一处理，有效去除了冗余决策，提升了模型的简洁性和实用性。

3) 实践相容性

通过查阅文献和之前的研究结果我们发现当次品率由抽样检测得到，实际次品率成正态分布；模拟实际的决策过程，剔除了不可能情况

4) 优化决策逻辑与简化模型

01 规划设置的编码和解码机制，能够高效地从数字状态映射到具体的生产决策，简化了决策过程并提高了模型的可操作性。通过剪枝操作简化了对可行情况的讨论，而**状态压缩**的方法又增强了模型的逻辑性与简洁性。

5.2 模型的不足

1) 计算复杂性：模型在处理多阶段决策和多组件系统时，涉及大量的状态组合和计算，这可能导致计算负担较重，尤其是在更大规模的生产环境中。

2) 静态模型假设：模型假设在决策过程中，市场条件和生产参数保持不变。然而，现实世界这些因素可能会随时间和市场动态而变化，模型未能完全捕捉这种动态性。

3) 局限性分析：本文提出的模型在当前设定的生产条件和假设下表现出较好的使用效果，但由于时间和资源的限制，未能对更多变化的生产环境进行广泛的测试和验证。因此，对于其他生产情形（如：小批量定制生产，高变异性原材料供应），模型可能无法实现同样高效的决策支持。

5.3 模型的推广

1) 广泛的行业适用性：

虽然本文的研究聚焦于电子产品制造，但模型的基本原理和方法论可以跨行业应用，如汽车制造、航空航天和医疗设备制造等领域。

2) 灵活的参数调整与定制化：

模型允许用户根据具体生产环境和业务需求，调整关键参数如次品率、检测成本、市场售价等，以实现最优的生产决策。

3) 模块化与可扩展性：

模型的模块化设计支持根据特定生产流程的需求，灵活添加、修改或删除模块，从而适应不同的生产环境和管理要求。

4) 技术整合与智能化：

模型可以与物联网(IoT)设备、大数据分析工具和人工智能算法等先进技术整合，实现数据的实时采集、分析和决策支持，提升生产过程的智能化水平。

参考文献

- [1] Gauss C F, Theoria motus corporum coelestium in sectionibus conicis solem ambientium[M]. FA Perthes, 1877.
- [2] 标准正态分布_百度百科,
<https://baike.baidu.com/item/%E6%A0%87%E5%87%86%E6%AD%A3%E6%80%81%E5%88%86%E5%B8%83/552653>, 2024.9.5
- [3] 佚名.算法导论[M].机械工业出版社,2013.

附录

附录 1

问题一模拟样本次品率随样本量变化 python 代码

```
1. import matplotlib.pyplot as plt
2. import numpy as np
3. import matplotlib
4.
5. # 设置 Matplotlib 的字体为支持中文的字体
6. matplotlib.rcParams['font.family'] = 'SimHei' # 'SimHei' 是黑体的字体名
7. matplotlib.rcParams['font.size'] = 12 # 可以设置字体大小
8. # 设置样本量范围
9. sample_sizes = np.arange(1, 1001, 5) # 从 1 到 1000, 步长为 5
10. p_0 = 0.1 # 假设标称次品率为 10%
11.
12. # 模拟次品率
13. np.random.seed(0) # 设置随机种子以获得可重复的结果
14. simulated_defect_rates = np.random.binomial(n=sample_sizes, p=p_0, size=sample_sizes.shape) / sample_sizes * 100
15.
16. # 绘制样本次品率随样本量变化的图
17. plt.figure(figsize=(10, 6))
18. plt.plot(sample_sizes, simulated_defect_rates, marker='o', linestyle='-', color='b', label='模拟次品率')
19. plt.axhline(12.00, color='r', linestyle='-', label='95%置信临界值 (12.00%)')
20. plt.axhline(8.00, color='g', linestyle='-', label='90%置信临界值 (8.00%)')
21. plt.title('样本次品率随样本量变化的模拟')
22. plt.xlabel('样本量')
23. plt.ylabel('次品率 (%)')
24. plt.legend()
25. plt.grid(True)
26. plt.show()
```

附录 2

问题二求决策模型代码

```
1. #include<iostream>
2. #include<set>
3. using namespace std;
4. //初始化零配件 1 和 2 的次品率, 购买单价, 检测成本以及成品的相关指标
5. double defective_rate_1[7] = {0, 0.1, 0.2, 0.1, 0.2, 0.1, 0.05};
6. double purchase_price_1[7] = {0, 4, 4, 4, 4, 4, 4};
7. double testing_cost_1[7] = {0, 2, 2, 2, 1, 8, 2};
8. double defective_rate_2[7] = {0, 0.1, 0.2, 0.1, 0.2, 0.2, 0.05};
9. double purchase_price_2[7] = {0, 18, 18, 18, 18, 18, 18};
10. double testing_cost_2[7] = {0, 3, 3, 3, 1, 1, 3};
11. double product_defective_rate[7] = {0, 0.1, 0.2, 0.1, 0.2, 0.1, 0.05};
12. double assembly_cost[7] = {0, 6, 6, 6, 6, 6, 6};
13. double product_testing_cost[7] = {0, 3, 3, 3, 2, 2, 3};
14. double market_price[7] = {0, 56, 56, 56, 56, 56, 56};
15. double dismantling_cost[7] = {0, 5, 5, 5, 5, 5, 40};
16. double exchange_losses[7] = {0, 6, 6, 30, 30, 10, 10};
17. double ans[7] = {0};
18. double quantity_1 = 10000, quantity_2 = 10000;
```

```

19. double result[7][16] = {0};
20. int ans_state[7] = {0};
21. int main() {
22.     for (int i = 1; i <= 6; i++) {
23.         for (int state = 0; state <= 15; state++) {
24.             double res = purchase_price_1[i] * quantity_1 + purchase_price_2[i] *
                quantity_2;
25.             if (state & 1) {
26.                 res += testing_cost_1[i] * quantity_1;
27.                 double now_quantity_1 = quantity_1 * (1 - defective_rate_1[i]);
28.                 if ((state >> 1) & 1) {
29.                     res += testing_cost_2[i] * quantity_2;
30.                     double now_quantity_2 = quantity_2 * (1 - defective_rate_2[i]);
31.                     double product_quantity = min(now_quantity_1, now_quantity_2);
32.                     res += product_quantity * assembly_cost[i];
33.                     if ((state >> 2) & 1) {
34.                         res += product_testing_cost[i] * product_quantity;
35.                         if ((state >> 3) & 1) {
36.                             res += product_quantity * product_defective_rate[i] *
                                (dismantling_cost[i] + product_testing_cost[i] + assembly_cost[i]) *
                                (1.0 / (1.0 - product_defective_rate[i]));
37.                             result[i][state] = product_quantity * market_price[i] -
                                res;
38.                         } else {
39.                             result[i][state] = product_quantity * (1 - product_defec-
                                tive_rate[i]) * market_price[i] - res;
40.                         }
41.                     } else {
42.                         if ((state >> 3) & 1) {
43.                             res += product_quantity * product_defective_rate[i] * (ex-
                                change_losses[i] + dismantling_cost[i] + assembly_cost[i]) * (1.0 / (1.0
                                - product_defective_rate[i]));
44.                             result[i][state] = product_quantity * market_price[i] -
                                res;
45.                         } else {
46.                             res += product_quantity * product_defective_rate[i] * ex-
                                change_losses[i];
47.                             result[i][state] = product_quantity * (1 - product_defec-
                                tive_rate[i]) * market_price[i] - res;
48.                         }
49.                     }
50.                 } else {
51.                     double product_quantity = min(now_quantity_1, quantity_2);
52.                     res += product_quantity * assembly_cost[i];
53.                     if ((state >> 2) & 1) {
54.                         res += product_testing_cost[i] * product_quantity;
55.
56.                         if (((state >> 3) & 1) == 0) {
57.                             result[i][state] = product_quantity * (1 - product_defec-
                                tive_rate[i] - defective_rate_2[i] + product_defective_rate[i] * defec-
                                tive_rate_2[i]) * market_price[i] - res;
58.                         }
59.                     } else {
60.
61.                         if (((state >> 3) & 1) == 0) {
62.                             res += product_quantity * (product_defective_rate[i] + de-
                                fective_rate_2[i] - product_defective_rate[i] * defective_rate_2[i]) *
                                exchange_losses[i];
63.                             result[i][state] = product_quantity * (1 - product_defec-
                                tive_rate[i] - defective_rate_2[i] + product_defective_rate[i] * defec-
                                tive_rate_2[i]) * market_price[i] - res;
64.                         }
65.                     }
66.                 }
67.             } else {

```

```

68.         if ((state >> 1) & 1) {
69.             res += testing_cost_2[i] * quantity_2;
70.             double now_quantity_2 = quantity_2 * (1 - defective_rate_2[i]);
71.             double product_quantity = min(quantity_1, now_quantity_2);
72.             res += product_quantity * assembly_cost[i];
73.             if ((state >> 2) & 1) {
74.                 res += product_testing_cost[i] * product_quantity;
75.                 if (((state >> 3) & 1) == 0) {
76.                     result[i][state] = product_quantity * (1 - product_defec-
77.                         tive_rate[i] - defective_rate_1[i] + product_defective_rate[i] * defec-
78.                         tive_rate_1[i]) * market_price[i] - res;
79.                 }
80.                 } else {
81.                     if (((state >> 3) & 1) == 0) {
82.                         res += product_quantity * (product_defective_rate[i] + de-
83.                             fective_rate_1[i] - product_defective_rate[i] * defective_rate_1[i]) *
84.                             exchange_losses[i];
85.                         result[i][state] = product_quantity * (1 - product_defec-
86.                             tive_rate[i] - defective_rate_1[i] + product_defective_rate[i] * defec-
87.                             tive_rate_1[i]) * market_price[i] - res;
88.                     }
89.                 }
90.             } else {
91.                 double product_quantity = min(quantity_1, quantity_2);
92.                 res += product_quantity * assembly_cost[i];
93.                 if ((state >> 2) & 1) {
94.                     res += product_testing_cost[i] * product_quantity;
95.                     if (((state >> 3) & 1) == 0) {
96.                         result[i][state] = product_quantity * (1 - (product_defec-
97.                             tive_rate[i] * (1 - defective_rate_1[i]) * (1 - defective_rate_2[i]) +
98.                             defective_rate_2[i] + defective_rate_1[i])) * market_price[i] - res;
99.                     }
100.                    } else {
101.                        if (((state >> 3) & 1) == 0) {
102.                            res += product_quantity * (product_defective_rate[i] * (1
103.                                - defective_rate_1[i]) * (1 - defective_rate_2[i]) + defective_rate_2[i]
104.                                + defective_rate_1[i]) * exchange_losses[i];
105.                            result[i][state] = product_quantity * (1 - (product_defec-
106.                                tive_rate[i] * (1 - defective_rate_1[i]) * (1 - defective_rate_2[i]) +
107.                                defective_rate_2[i] + defective_rate_1[i])) * market_price[i] - res;
108.                        }
109.                    }
110.                }
111.            }
112.        }
113.    }
114.    for (int i = 1; i <= 6; i++) {
115.        for (int j = 0; j <= 15; j++) {
116.            if (ans[i] < result[i][j]) ans[i] = result[i][j], ans_state[i] = j;
117.            //cout << result[i][j] << " ";
118.        }
119.        //cout << '\n';
120.    }
121.    for (int i = 1; i <= 6; i++) cout << ans[i] << " " << ans_state[i] << '\n';
122.    return 0;
123. }

```

附录 3

问题三决策模型求解代码

```
1. #include<iostream>
2. #include<string.h>
3. #include<set>
4. using namespace std;
5. set<int>s1={2,3,6,7,10,11,14,15,18,19,22,23,26,27};
6. set<int>s2={2,3,6,7,10,11};
7. //半成品结构体
8. struct semi_finished {
9.     int state;
10.    double quantity;
11.    double defect_rate;
12.    double cost;
13. } semi_finished_1[32] = {0, 0, 0, 0}, semi_finished_2[32] = {0, 0, 0, 0}, semi_finished_3[16] = {0, 0, 0, 0};
14. //成品结构体
15. struct Product {
16.    int state1, state2, state3, state4;
17.    double profit;
18. } product[32 * 32 * 16 * 32] = {-1, -1, -1, -1, 0};
19. int num = 32 * 32 * 16 * 32 - 1;
20. bool check(int x, double ans);
21. //计算半成品费用
22. void calculate_semi_cost(double part_defective_rate[], double part_purchase_price[], double part_testing_cost[], double product_defective_rate, double product_testing_cost, double assembly_cost, double dismantling_cost, struct semi_finished semi[], bool flag);
23. //计算成品费用
24. void calculate_product_cost(double part_defective_rate[], double part_purchase_price[], double part_quantity[], double part_testing_cost[], int part_state[], double product_defective_rate, double product_testing_cost, double assembly_cost, double dismantling_cost, double market_price, double exchange_losses);
25. //计算半成品检测和拆解费用
26. void calculate_temp_cost(int state, double quantity, double defect_rate, double product_defect_rate, double testing_cost, double dismantling, double assembly, struct semi_finished semi[], double cost, bool flag);
27. //计算成品检测和拆解费用
28. void calculate_temp_cost2(int state, double quantity, double defect_rate, double product_defect_rate, double testing_cost, double dismantling, double assembly, double cost, double market_price, double exchange_losses);
29. //初始化数据
30. double part_defect_rate[3][3] = {0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0};
31. double part_purchase_price[3][3] = {2, 8, 12, 2, 8, 12, 8, 12, 0};
32. double part_testing_cost[3][3] = {1, 1, 2, 1, 1, 2, 1, 2, 0};
33. double semi_defect_rate[3] = {0.1, 0.1, 0.1};
34. double semi_testing_cost[3] = {4, 4, 4};
35. double semi_assembly_cost[3] = {8, 8, 8};
36. double semi_dismantling_cost[3] = {6, 6, 6};
37. double product_defect_rate = 0.1;
38. double product_assembly_cost = 8;
39. double product_testing_cost = 6;
40. double product_dismantling_cost = 10;
41. double market_price = 200;
42. double exchange_losses = 40;
43. int main() {
44.    //分别计算半成品 1,2,3 的费用
```

```

47. calculate_semi_cost(part_defect_rate[0], part_purchase_price[0], part_test-
    ing_cost[0], semi_defect_rate[0], semi_testing_cost[0], semi_assembly_cost[0],
    semi_dismantling_cost[0], semi_finished_1, 0);
48. calculate_semi_cost(part_defect_rate[1], part_purchase_price[1], part_test-
    ing_cost[1], semi_defect_rate[1], semi_testing_cost[1], semi_assembly_cost[1],
    semi_dismantling_cost[1], semi_finished_2, 0);
49. calculate_semi_cost(part_defect_rate[2], part_purchase_price[2], part_test-
    ing_cost[2], semi_defect_rate[2], semi_testing_cost[2], semi_assembly_cost[2],
    semi_dismantling_cost[2], semi_finished_3, 1);
50. //遍历所有半成品情况组合，找出最优解
51. for (int i = 0; i <= 31; i++) {
52.     for (int j = 0; j <= 31; j++) {
53.         for (int k = 0; k <= 15; k++) {
54.             double temp_defect_rate[3] = {semi_finished_1[i].defect_rate, semi_fin-
    ished_2[j].defect_rate, semi_finished_3[k].defect_rate};
55.             double temp_purchase_price[3] = {semi_finished_1[i].cost, semi_fin-
    ished_2[j].cost, semi_finished_3[k].cost};
56.             double temp_quantity[3] = {semi_finished_1[i].quantity, semi_fin-
    ished_2[j].quantity, semi_finished_3[k].quantity};
57.             int temp_state[3] = {i, j, k};
58.             calculate_product_cost(temp_defect_rate, temp_purchase_price,
    temp_quantity, semi_testing_cost, temp_state, product_defect_rate, product_test-
    ing_cost, product_assembly_cost, product_dismantling_cost, market_price, ex-
    change_losses);
59.             for (int l = num + 1; l <= num + 32; l++) product[l].state1 = i, prod-
    uct[l].state2 = j, product[l].state3 = k;
60.         }
61.     }
62. }
63. double ans = 0;
64. int s1, s2, s3, s4;
65. for (int i = 0; i < 32 * 32 * 32 * 16; i++)
66.     {if (check(i,ans)) {ans = product[i].profit, s1 = product[i].state1, s2 = prod-
    uct[i].state2, s3 = product[i].state3, s4 = product[i].state4;
67.         //cout<<product[i].profit<<" "<<product[i].state1<<" "<<product[i].state2<<"
    "<<product[i].state3<<" "<<product[i].state4<<"\n";
68.     }
69. }
70. cout << ans << " " << s1 << " " << s2 << " " << s3 << " " << s4;
71. return 0;
72. }
73. bool check(int x,double ans)
74. {
75.     if(s1.count(product[x].state1)) return 0;
76.     if(product[x].state1!=product[x].state2) return 0;
77.     if(s2.count(product[x].state3)) return 0;
78.     if(ans>product[x].profit) return 0;
79.     if((product[x].state4&2)&&((product[x].state1&1)==0||((prod-
    uct[x].state2&1)==0||((product[x].state3&1)==0))) return 0;
80.     return 1;
81. }
82. void calculate_semi_cost(double part_defective_rate[], double part_purchase_price[],
    double part_testing_cost[], double product_defective_rate, double product_test-
    ing_cost, double assembly_cost, double dismantling_cost, struct semi_finished semi[],
    bool flag) {
83.     for (int state = 0; state <= 31; state++) {
84.         double now_quantity = 1;
85.         double cost = part_purchase_price[0] + part_purchase_price[1] + part_pur-
    chase_price[2];
86.         double now_defect_rate = product_defective_rate;
87.         if ((state >> 2) & 1 && (state >> 3) & 1 && (state >> 4) & 1) {
88.             cost += part_testing_cost[0] + part_testing_cost[1] + part_testing_cost[2];
89.             now_quantity = min((1 - part_defective_rate[0]), min( (1 - part_defec-
    tive_rate[1]), (1 - part_defective_rate[2])));
90.         }

```

```

91.     if (((state >> 2) & 1) == 0 && (state >> 3) & 1 && (state >> 4) & 1) {
92.         cost += part_testing_cost[1] + part_testing_cost[2];
93.         now_quantity = 1 - max(part_defective_rate[1], part_defective_rate[2]);
94.         now_defect_rate = now_defect_rate + part_defective_rate[0] - now_de-
fect_rate * part_defective_rate[0];
95.     }
96.     if (((state >> 2) & 1 && ((state >> 3) & 1) == 0 && (state >> 4) & 1) {
97.         cost += part_testing_cost[0] + part_testing_cost[2];
98.         now_quantity = 1 - max(part_defective_rate[0], part_defective_rate[2]);
99.         now_defect_rate = now_defect_rate + part_defective_rate[1] - now_de-
fect_rate * part_defective_rate[1];
100.    }
101.    if (((state >> 2) & 1) == 0 && ((state >> 3) & 1) == 0 && (state >> 4) & 1) {
102.        cost += part_testing_cost[2];
103.        now_defect_rate = 1 - (1 - part_defective_rate[1]) * (1 - part_defec-
tive_rate[0]) * (1 - product_defective_rate);
104.        now_quantity = 1 - part_defective_rate[2];
105.    }
106.    if ((state >> 2) & 1 && (state >> 3) & 1 && ((state >> 4) & 1) == 0) {
107.        cost += part_testing_cost[1] + part_testing_cost[0];
108.        now_quantity = 1 - max(part_defective_rate[1], part_defective_rate[0]);
109.        now_defect_rate = now_defect_rate + part_defective_rate[2] - now_de-
fect_rate * part_defective_rate[2];
110.    }
111.    if (((state >> 2) & 1) == 0 && (state >> 3) & 1 && ((state >> 4) & 1) == 0)
{
112.        cost += part_testing_cost[1];
113.        now_defect_rate = 1 - (1 - part_defective_rate[2]) * (1 - part_defec-
tive_rate[0]) * (1 - product_defective_rate);
114.        now_quantity = 1 - part_defective_rate[1];
115.    }
116.    if ((state >> 2) & 1 && ((state >> 3) & 1) == 0 && ((state >> 4) & 1) == 0) {
117.        cost += part_testing_cost[0];
118.        now_defect_rate = 1 - (1 - part_defective_rate[1]) * (1 - part_defec-
tive_rate[2]) * (1 - product_defective_rate);
119.        now_quantity = 1 - part_defective_rate[0];
120.    }
121.    if (((state >> 2) & 1) == 0 && ((state >> 3) & 1) == 0 && ((state >> 4) & 1)
== 0) {
122.        now_defect_rate = 1 - (1 - part_defective_rate[0]) * (1 - part_defec-
tive_rate[1]) * (1 - part_defective_rate[2]) * (1 - product_defective_rate);
123.    }
124.    calculate_temp_cost(state, now_quantity, now_defect_rate, product_defec-
tive_rate, product_testing_cost, dismantling_cost, assembly_cost, semi, cost, flag);
125. }
126. }
127. void calculate_product_cost(double part_defective_rate[], double part_pur-
chase_price[], double part_quantity[],
128.     double part_testing_cost[], int part_state[], double prod-
uct_defective_rate, double product_testing_cost,
129.     double assembly_cost, double dismantling_cost, double mar-
ket_price, double exchange_losses) {
130.     for (int state = 0; state <= 31; state++) {
131.         if ((part_state[0] & 1 && ((state >> 2) & 1) == 0) || (part_state[1] & 1 &&
((state >> 3) & 1) == 0) || (part_state[2] & 1 && ((state >> 4) & 1) == 0)) {
132.             num--;
133.             continue;
134.         }
135.         double now_quantity = 1;
136.         double cost = part_purchase_price[0] + part_purchase_price[1] + part_pur-
chase_price[2];
137.         double now_defect_rate = product_defective_rate;
138.         if ((state >> 2) & 1 && (state >> 3) & 1 && (state >> 4) & 1) {
139.             cost += ((part_state[0] & 1) == 0 ? part_testing_cost[0] * part_quan-
tity[0] : 0 +

```

```

140.          ((part_state[1]) & 1) == 0 ? part_testing_cost[1] * part_quantity[1] : 0 +
141.          ((part_state[2]) & 1) == 0 ? part_testing_cost[2] * part_quantity[2] : 0;
142.      now_quantity = min(part_quantity[0] * (1 - part_defective_rate[0]),
143.      min(part_quantity[1] * (1 - part_defective_rate[1]), part_quantity[2] * (1 - part_defective_rate[2])));
144.      if (((state >> 2) & 1) == 0 && (state >> 3) & 1 && (state >> 4) & 1) {
145.          cost +=
146.          ((part_state[1]) & 1) == 0 ? part_testing_cost[1] * part_quantity[1] :
147.          0 +
148.          ((part_state[2]) & 1) == 0 ? part_testing_cost[2] * part_quantity[2] :
149.          0;
150.          now_quantity = min(part_quantity[0], min(part_quantity[1] * (1 - part_defective_rate[1]), part_quantity[2] * (1 - part_defective_rate[2])));
151.          now_defect_rate = now_defect_rate + part_defective_rate[0] - now_defect_rate * part_defective_rate[0];
152.      }
153.      if (((state >> 2) & 1 && ((state >> 3) & 1) == 0 && (state >> 4) & 1) {
154.          cost += ((part_state[0]) & 1) == 0 ? part_testing_cost[0] * part_quantity[0] : 0 +
155.          ((part_state[2]) & 1) == 0 ? part_testing_cost[2] * part_quantity[2] : 0;
156.          now_quantity = min(part_quantity[1], min(part_quantity[0] * (1 - part_defective_rate[0]), part_quantity[2] * (1 - part_defective_rate[2])));
157.          now_defect_rate = now_defect_rate + part_defective_rate[1] - now_defect_rate * part_defective_rate[1];
158.      }
159.      if (((state >> 2) & 1) == 0 && ((state >> 3) & 1) == 0 && (state >> 4) & 1) {
160.          cost +=
161.          ((part_state[2]) & 1) == 0 ? part_testing_cost[2] * part_quantity[2] : 0;
162.          now_defect_rate = 1 - (1 - part_defective_rate[1]) * (1 - part_defective_rate[0]) * (1 - product_defective_rate);
163.          now_quantity = min(part_quantity[0], min(part_quantity[1], part_quantity[2] * (1 - part_defective_rate[2])));
164.      }
165.      if ((state >> 2) & 1 && (state >> 3) & 1 && ((state >> 4) & 1) == 0) {
166.          cost += ((part_state[0]) & 1) == 0 ? part_testing_cost[0] * part_quantity[0] : 0 +
167.          ((part_state[1]) & 1) == 0 ? part_testing_cost[1] * part_quantity[1] : 0;
168.          now_quantity = min(part_quantity[2], min(part_quantity[1] * (1 - part_defective_rate[1]), part_quantity[0] * (1 - part_defective_rate[0])));
169.          now_defect_rate = now_defect_rate + part_defective_rate[2] - now_defect_rate * part_defective_rate[2];
170.      }
171.      if (((state >> 2) & 1) == 0 && (state >> 3) & 1 && ((state >> 4) & 1) == 0) {
172.          cost +=
173.          ((part_state[1]) & 1) == 0 ? part_testing_cost[1] * part_quantity[1] : 0;
174.          now_defect_rate = 1 - (1 - part_defective_rate[2]) * (1 - part_defective_rate[0]) * (1 - product_defective_rate);
175.          now_quantity = min(part_quantity[0], min(part_quantity[2], part_quantity[1] * (1 - part_defective_rate[1])));
176.      }
177.      if ((state >> 2) & 1 && ((state >> 3) & 1) == 0 && ((state >> 4) & 1) == 0) {
178.          cost += ((part_state[0]) & 1) == 0 ? part_testing_cost[0] * part_quantity[0] : 0;
179.          now_defect_rate = 1 - (1 - part_defective_rate[1]) * (1 - part_defective_rate[2]) * (1 - product_defective_rate);

```

```

179.     now_quantity = min(part_quantity[1], min(part_quantity[2], part_quantity[0] * (1 - part_defective_rate[0])));
180. }
181.     if (((state >> 2) & 1) == 0 && ((state >> 3) & 1) == 0 && ((state >> 4) & 1) == 0) {
182.         now_quantity = min(part_quantity[0], min(part_quantity[1], part_quantity[2]));
183.         now_defect_rate = 1 - (1 - part_defective_rate[0]) * (1 - part_defective_rate[1]) * (1 - part_defective_rate[2]) * (1 - product_defective_rate);
184.     }
185.     calculate_temp_cost2(state, now_quantity, now_defect_rate, product_defective_rate, product_testing_cost, dismantling_cost, assembly_cost, cost, market_price, exchange_losses);
186. }
187. }
188. void calculate_temp_cost(int state, double quantity, double defect_rate, double product_defect_rate, double testing_cost, double dismantling, double assembly, struct semi_finished semi[], double cost, bool flag) {
189.     cost += quantity * assembly;
190.     if (state & 1 && (state >> 1) & 1 && (((state >> 2) & 7) == 7 || (((state >> 2) & 3) == 3 && flag))) {
191.         cost += quantity * testing_cost;
192.         cost += quantity * defect_rate * (dismantling + testing_cost + assembly) * (1 / (1 - product_defect_rate));
193.         defect_rate = 0;
194.         semi[state] = {state, quantity, defect_rate, cost};
195.     }
196.     if ((state & 1) == 0 && (state >> 1) & 1 && (((state >> 2) & 7) == 7 || (((state >> 2) & 3) == 3 && flag))) {
197.         semi[state] = {state, quantity, defect_rate, cost};
198.     }
199.     if (state & 1 && ((state >> 1) & 1) == 0) {
200.         cost += quantity * testing_cost;
201.         quantity *= (1 - defect_rate);
202.         defect_rate = 0;
203.         semi[state] = {state, quantity, defect_rate, cost};
204.     }
205.     if ((state & 1) == 0 && ((state >> 1) & 1) == 0) {
206.         semi[state] = {state, quantity, defect_rate, cost};
207.     }
208. }
209. void calculate_temp_cost2(int state, double quantity, double defect_rate, double product_defect_rate, double testing_cost, double dismantling, double assembly, double cost, double market_price, double exchange_losses) {
210.     cost += quantity * assembly;
211.     if (state & 1 && (state >> 1) & 1 && (((state >> 2) & 7) == 7)) {
212.         cost += quantity * testing_cost;
213.         cost += quantity * defect_rate * (dismantling + testing_cost + assembly) * (1 / (1 - product_defect_rate));
214.         product[num].state4 = state & 3;
215.         product[num].profit = quantity * market_price - cost;
216.     }
217.     if ((state & 1) == 0 && (state >> 1) & 1 && (((state >> 2) & 7) == 7)) {
218.         cost += quantity * defect_rate * (dismantling + exchange_losses + assembly) * (1 / (1 - product_defect_rate));
219.         product[num].state4 = state & 3;
220.         product[num].profit = quantity * market_price - cost;
221.     }
222.     if (state & 1 && ((state >> 1) & 1) == 0) {
223.         cost += quantity * testing_cost;
224.         quantity *= (1 - defect_rate);
225.         product[num].state4 = state & 3;
226.         product[num].profit = quantity * market_price - cost;
227.     }
228.     if ((state & 1) == 0 && ((state >> 1) & 1) == 0) {

```



```

229.     cost += quantity * defect_rate * exchange_losses;
230.     product[num].state4 = state & 3;
231.     product[num].profit = quantity * (1 - defect_rate) * market_price - cost;
232. }
233. num--;
234. }

```

附录 4

问题四决策模型求解代码

```

1. #include<iostream>
2. #include<set>
3. #include<math.h>
4. using namespace std;
5. //初始化零配件 1 和 2 的次品率, 购买单价, 检测成本以及成品的相关指标
6. double defective_rate_1[7] = { 0, 0.1, 0.2, 0.1, 0.2, 0.1, 0.05 };
7. double purchase_price_1[7] = { 0, 4, 4, 4, 4, 4, 4 };
8. double testing_cost_1[7] = { 0, 2, 2, 2, 1, 8, 2 };
9. double defective_rate_2[7] = { 0, 0.1, 0.2, 0.1, 0.2, 0.2, 0.05 };
10. double purchase_price_2[7] = { 0, 18, 18, 18, 18, 18, 18 };
11. double testing_cost_2[7] = { 0, 3, 3, 3, 1, 1, 3 };
12. double product_defective_rate[7] = { 0, 0.1, 0.2, 0.1, 0.2, 0.1, 0.05 };
13. double assembly_cost[7] = { 0, 6, 6, 6, 6, 6, 6 };
14. double product_testing_cost[7] = { 0, 3, 3, 3, 2, 2, 3 };
15. double market_price[7] = { 0, 56, 56, 56, 56, 56, 56 };
16. double dismantling_cost[7] = { 0, 5, 5, 5, 5, 5, 40 };
17. double exchange_losses[7] = { 0, 6, 6, 30, 30, 10, 10 };
18. double ans[7] = { 0 };
19. double quantity_1 = 10000, quantity_2 = 10000;
20. double result[7][16] = { 0 };
21. int ans_state[7] = { 0 };
22. int main() {
23.     int row = 0;
24.     for (int i = 1; i <= 6; i++) {
25.         for (int state = 0; state <= 15; state++) {
26.             double res = purchase_price_1[i] * quantity_1 + purchase_price_2[i] * quantity_2;
27.             if (state & 1) {
28.                 res += testing_cost_1[i] * quantity_1;
29.                 double now_quantity_1 = quantity_1 * (1 - defective_rate_1[i]);
30.                 if ((state >> 1) & 1) {
31.                     res += testing_cost_2[i] * quantity_2;
32.                     double now_quantity_2 = quantity_2 * (1 - defective_rate_2[i]);
33.                     double product_quantity = min(now_quantity_1, now_quantity_2);
34.                     res += product_quantity * assembly_cost[i];
35.                     if ((state >> 2) & 1) {
36.                         res += product_testing_cost[i] * product_quantity;
37.                         if ((state >> 3) & 1) {
38.                             res += product_quantity * product_defective_rate[i] * (dismantling_cost[i] + product_testing_cost[i] + assembly_cost[i]) * (1.0 / (1.0 - product_defective_rate[i]));
39.                             result[i][state] = product_quantity * market_price[i] - res;
40.                         }
41.                     }
42.                     result[i][state] = product_quantity * (1 - product_defective_rate[i]) * market_price[i] - res;
43.                 }
44.             }
45.             else {
46.                 if ((state >> 3) & 1) {

```

```

47.         res += product_quantity * product_defective_rate[i] * (ex-
change_losses[i] + dismantling_cost[i] + assembly_cost[i]) * (1.0 / (1.0 - product_de-
fective_rate[i]));
48.         result[i][state] = product_quantity * market_price[i] - res;
49.     }
50.     else {
51.         res += product_quantity * product_defective_rate[i] * ex-
change_losses[i];
52.         result[i][state] = product_quantity * (1 - product_defec-
tive_rate[i]) * market_price[i] - res;
53.     }
54. }
55. }
56. else {
57.     double product_quantity = min(now_quantity_1, quantity_2);
58.     res += product_quantity * assembly_cost[i];
59.     if ((state >> 2) & 1) {
60.         res += product_testing_cost[i] * product_quantity;
61.
62.         if (((state >> 3) & 1) == 0) {
63.             result[i][state] = product_quantity * (1 - product_defec-
tive_rate[i] - defective_rate_2[i] + product_defective_rate[i] * defective_rate_2[i])
* market_price[i] - res;
64.         }
65.     }
66.     else {
67.
68.         if (((state >> 3) & 1) == 0) {
69.             res += product_quantity * (product_defective_rate[i] + de-
fective_rate_2[i] - product_defective_rate[i] * defective_rate_2[i]) * ex-
change_losses[i];
70.             result[i][state] = product_quantity * (1 - product_defec-
tive_rate[i] - defective_rate_2[i] + product_defective_rate[i] * defective_rate_2[i])
* market_price[i] - res;
71.         }
72.     }
73. }
74. }
75. else {
76.     if ((state >> 1) & 1) {
77.         res += testing_cost_2[i] * quantity_2;
78.         double now_quantity_2 = quantity_2 * (1 - defective_rate_2[i]);
79.         double product_quantity = min(quantity_1, now_quantity_2);
80.         res += product_quantity * assembly_cost[i];
81.         if ((state >> 2) & 1) {
82.             res += product_testing_cost[i] * product_quantity;
83.             if (((state >> 3) & 1) == 0) {
84.                 result[i][state] = product_quantity * (1 - product_defec-
tive_rate[i] - defective_rate_1[i] + product_defective_rate[i] * defective_rate_1[i])
* market_price[i] - res;
85.             }
86.         }
87.         else {
88.             if (((state >> 3) & 1) == 0) {
89.                 res += product_quantity * (product_defective_rate[i] + de-
fective_rate_1[i] - product_defective_rate[i] * defective_rate_1[i]) * ex-
change_losses[i];
90.                 result[i][state] = product_quantity * (1 - product_defec-
tive_rate[i] - defective_rate_1[i] + product_defective_rate[i] * defective_rate_1[i])
* market_price[i] - res;
91.             }
92.         }
93.     }
94.     else {
95.         double product_quantity = min(quantity_1, quantity_2);

```

```

96.         res += product_quantity * assembly_cost[i];
97.         if ((state >> 2) & 1) {
98.             res += product_testing_cost[i] * product_quantity;
99.             if (((state >> 3) & 1) == 0) {
100.                 result[i][state] = product_quantity * (1 - (product_defec-
    tive_rate[i] * (1 - defective_rate_1[i]) * (1 - defective_rate_2[i]) + defec-
    tive_rate_2[i] + defective_rate_1[i])) * market_price[i] - res;
101.             }
102.         }
103.         else {
104.             if (((state >> 3) & 1) == 0) {
105.                 res += product_quantity * (product_defective_rate[i] * (1 -
    defective_rate_1[i]) * (1 - defective_rate_2[i]) + defective_rate_2[i] + defec-
    tive_rate_1[i]) * exchange_losses[i];
106.                 result[i][state] = product_quantity * (1 - (product_defec-
    tive_rate[i] * (1 - defective_rate_1[i]) * (1 - defective_rate_2[i]) + defec-
    tive_rate_2[i] + defective_rate_1[i])) * market_price[i] - res;
107.             }
108.         }
109.     }
110. }
111. }
112. }
113. set<int>s;
114. s.insert({ 8,9,10,12,13,14 });
115. for (int i = 1; i <= 6; i++) {
116.     for (int j = 0; j <= 15; j++) {
117.         if (ans[i] < result[i][j] && !s.count(j)) ans[i] = result[i][j],
    ans_state[i] = j;
118.         //cout << result[i][j] << " ";
119.     }
120.     //cout << '\n';
121. }
122. //for (int i = 1; i <= 6; i++) cout << ans[i] << " " << ans_state[i] << '\n';
123.
124. double dr1[7] = { 0, 0.1, 0.2, 0.1, 0.2, 0.1, 0.05 };
125. double dr2[7] = { 0, 0.1, 0.2, 0.1, 0.2, 0.1, 0.05 };
126. double cr[7] = { 0, 0.1, 0.2, 0.1, 0.2, 0.1, 0.05 };
127. for (int i = 1; i <= 6; i++) {
128.     cout << "情况" << i << endl;
129.     for (defective_rate_1[i] = dr1[i] - 1.96*sqrt(dr1[i]*(1-dr1[i])/609); defec-
    tive_rate_1[i] <= dr1[i] + 1.25*1.96*sqrt(dr1[i]*(1-dr1[i])/609); defective_rate_1[i]
    = defective_rate_1[i] + 0.25*1.96*sqrt(dr1[i]*(1-dr1[i])/609)) {
130.         for (defective_rate_2[i] = dr2[i] - 1.96*sqrt(dr1[i]*(1-dr1[i])/609); de-
    fective_rate_2[i] <= dr2[i] + 1.25*1.96*sqrt(dr1[i]*(1-dr1[i])/609); defec-
    tive_rate_2[i] += 0.25*1.96*sqrt(dr1[i]*(1-dr1[i])/609)) {
131.             for (product_defective_rate[i] = cr[i] - 1.96*sqrt(dr1[i]*(1-
    dr1[i])/609); product_defective_rate[i] <= cr[i] + 1.25*1.96*sqrt(dr1[i]*(1-
    dr1[i])/609); product_defective_rate[i] += 0.25*1.96*sqrt(dr1[i]*(1-dr1[i])/609))
132.             {
133.                 for (int state = 0; state <= 15; state++) {
134.                     double res = purchase_price_1[i] * quantity_1 + pur-
    chase_price_2[i] * quantity_2;
135.                     if (state & 1) {
136.                         res += testing_cost_1[i] * quantity_1;
137.                         double now_quantity_1 = quantity_1 * (1 - defec-
    tive_rate_1[i]);
138.                         if ((state >> 1) & 1) {
139.                             res += testing_cost_2[i] * quantity_2;
140.                             double now_quantity_2 = quantity_2 * (1 - defec-
    tive_rate_2[i]);
141.                             double product_quantity = min(now_quantity_1, now_qun-
    tity_2);
142.                             res += product_quantity * assembly_cost[i];
143.                             if ((state >> 2) & 1) {

```

```

144.         res += product_testing_cost[i] * product_quantity;
145.         if ((state >> 3) & 1) {
146.             res += product_quantity * product_defec-
147. tive_rate[i] * (dismantling_cost[i] + product_testing_cost[i] + assembly_cost[i]) *
(1.0 / (1.0 - product_defective_rate[i]));
148.             result[i][state] = product_quantity * mar-
ket_price[i] - res;
149.         }
150.         else {
151.             result[i][state] = product_quantity * (1 - prod-
uct_defective_rate[i]) * market_price[i] - res;
152.         }
153.         else {
154.             if ((state >> 3) & 1) {
155.                 res += product_quantity * product_defec-
tive_rate[i] * (exchange_losses[i] + dismantling_cost[i] + assembly_cost[i]) * (1.0 /
(1.0 - product_defective_rate[i]));
156.                 result[i][state] = product_quantity * mar-
ket_price[i] - res;
157.             }
158.             else {
159.                 res += product_quantity * product_defec-
tive_rate[i] * exchange_losses[i];
160.                 result[i][state] = product_quantity * (1 - prod-
uct_defective_rate[i]) * market_price[i] - res;
161.             }
162.         }
163.     }
164.     else {
165.         double product_quantity = min(now_quantity_1, quan-
tity_2);
166.         res += product_quantity * assembly_cost[i];
167.         if ((state >> 2) & 1) {
168.             res += product_testing_cost[i] * product_quantity;
169.         }
170.         if (((state >> 3) & 1) == 0) {
171.             result[i][state] = product_quantity * (1 - prod-
uct_defective_rate[i] - defective_rate_2[i] + product_defective_rate[i] * defec-
tive_rate_2[i]) * market_price[i] - res;
172.         }
173.         }
174.         else {
175.             if (((state >> 3) & 1) == 0) {
176.                 res += product_quantity * (product_defec-
tive_rate[i] + defective_rate_2[i] - product_defective_rate[i] * defective_rate_2[i])
* exchange_losses[i];
177.                 result[i][state] = product_quantity * (1 - prod-
uct_defective_rate[i] - defective_rate_2[i] + product_defective_rate[i] * defec-
tive_rate_2[i]) * market_price[i] - res;
178.             }
179.         }
180.     }
181. }
182. }
183. else {
184.     if ((state >> 1) & 1) {
185.         res += testing_cost_2[i] * quantity_2;
186.         double now_quantity_2 = quantity_2 * (1 - defec-
tive_rate_2[i]);
187.         double product_quantity = min(quantity_1, now_quan-
tity_2);
188.         res += product_quantity * assembly_cost[i];
189.         if ((state >> 2) & 1) {
190.             res += product_testing_cost[i] * product_quantity;

```

```

191.         if (((state >> 3) & 1) == 0) {
192.             result[i][state] = product_quantity * (1 - prod-
uct_defective_rate[i] - defective_rate_1[i] + product_defective_rate[i] * defec-
tive_rate_1[i]) * market_price[i] - res;
193.         }
194.     }
195.     else {
196.         if (((state >> 3) & 1) == 0) {
197.             res += product_quantity * (product_defec-
tive_rate[i] + defective_rate_1[i] - product_defective_rate[i] * defective_rate_1[i])
* exchange_losses[i];
198.             result[i][state] = product_quantity * (1 - prod-
uct_defective_rate[i] - defective_rate_1[i] + product_defective_rate[i] * defec-
tive_rate_1[i]) * market_price[i] - res;
199.         }
200.     }
201. }
202. else {
203.     double product_quantity = min(quantity_1, quantity_2);
204.     res += product_quantity * assembly_cost[i];
205.     if ((state >> 2) & 1) {
206.         res += product_testing_cost[i] * product_quantity;
207.         if (((state >> 3) & 1) == 0) {
208.             result[i][state] = product_quantity * (1 -
(product_defective_rate[i] * (1 - defective_rate_1[i]) * (1 - defective_rate_2[i]) +
defective_rate_2[i] + defective_rate_1[i])) * market_price[i] - res;
209.         }
210.     }
211.     else {
212.         if (((state >> 3) & 1) == 0) {
213.             res += product_quantity * (product_defec-
tive_rate[i] * (1 - defective_rate_1[i]) * (1 - defective_rate_2[i]) + defec-
tive_rate_2[i] + defective_rate_1[i]) * exchange_losses[i];
214.             result[i][state] = product_quantity * (1 -
(product_defective_rate[i] * (1 - defective_rate_1[i]) * (1 - defective_rate_2[i]) +
defective_rate_2[i] + defective_rate_1[i])) * market_price[i] - res;
215.         }
216.     }
217. }
218. }
219. }
220.
221.     for (int k = 1; k <= 6; k++) {
222.         ans[k] = -9999999;
223.     }
224.     for (int i = 1; i <= 6; i++) {
225.         for (int j = 0; j <= 15; j++) {
226.             if (ans[i] < result[i][j] && !s.count(j)) ans[i] = re-
sult[i][j], ans_state[i] = j;
227.         }
228.     }
229. }
230.     cout << ans_state[i] << " ";
231.
232. }
233.
234.
235.     }
236.     cout << endl;
237. }
238.     cout << '\n';
239.     cout << endl;
240. }
241.
242.

```

```
243.     return 0;  
244. }
```